

Семинар №6

Аллокаторы

Зачем нужны кастомные аллокаторы

- Хотим более гибко и таргетированно менеджить память
- Хотим использовать свои эвристики выделения памяти
- Хотим реже обращаться к стандартному аллокатору

Синтаксис

- `allocate(n)` – выделить память под `n` элементов типа `T`(, но не создавать их)
- `deallocate(p, n)` – освободить память от `n` объектов типа `T` по указателю `p`(, но не уничтожать объекты)
- `construct(xp, args...)` – создать объект по адресу `xp` из аргументов конструктора `args`(, но память уже выделена)
- `destroy(xp)` – разрушить объект по указателю `xp`(, но не освобождать память)

allocator_traits

- Автоматически доопределяет очевидные вещи для аллокаторов (rebind, construct, destroy, using...)
- Эти методы и using-и можно не прописывать в своём аллокаторе, если они тривиальны
- Обращаться к методам аллокатора нужно всегда через allocator_traits

Allocator-aware containers

- Что делать с аллокатором контейнера при его использовании в конструкторе копирования? (например, скопировать аллокатор или использовать дефолтный) – отвечает **select_on_container_copy_construction**
- А операторе присваивания копированием? - **propagate_on_container_copy_assignment**, заменяет аллокатор, если этот флаг выставлен.

Как хранить аллокатор?

- Дефолтный аллокатор пустой, поэтому можно отнаследоваться от аллокатора, тогда применится EBO(empty base optimization)
- Начиная с C++20 можно использовать [атрибут](#) `[[no_unique_address]]`

Scoped allocators

- Пусть у нас есть контейнер для хранения других контейнеров
- Как заставить внутренние контейнеры использовать тот же аллокатор, что и внешний контейнер?
- `scoped_allocator_adapter`
- В таком случае, внутренний контейнер получит нужный ему аллокатор в качестве последнего аргумента в `construct`
- Если аллокаторов больше, чем уровней контейнера, то они не используются, иначе – используется последний тип аллокатора
- Должны проверить, есть ли у объекта вообще конструктор от аллокатора ([`std::uses\_allocator`](#))

Polymorphic allocators

- При работе с аллокаторами могут возникнуть проблемы из-за того, что аллокатор является шаблонным параметром
- Например, нельзя присваивать другому контейнеру с другим аллокатором, или передавать в функцию
- Эту проблему решают полиморфные аллокаторы из `std::pmr`
- Нужно использовать контейнеры из `std::pmr`, например, `std::pmr::vector<char>`
- Автоматически поддерживают функционал `scoped_allocator`
- Позволяют спрятать реальный тип аллокатора с помощью динамического полиморфизма (как именно – изучим позже)
- `std::memory_resource` – абстрактный класс